

Day 16: Mask Account Number Using Slicing

1. Day Number

Day 16

2. Topic Name

String Indexing and Slicing

3. Connection

Earlier, you stored an account number.

Today, you will learn how to protect that account number by displaying only its **last 4 digits**.

For example:

```
Original account number: 1234567890  
Masked account number:  XXXXXX7890
```

This is called **masking sensitive information**.

4. Important Topics

- String indexing
 - String slicing
 - Negative indexing
 - String length using `len()`
 - String repetition using `*`
-

5. Foundational Notes

What is a string?

A string is text written inside quotes.

```
account_number = "1234567890"
```

Even though the account number contains digits, keeping it as a string is useful because:

- You do not perform calculations on it.
- It may contain leading zeros.
- String slicing works directly on it.

For example:

```
account_number = "0012345678"
```

If converted to an integer, the starting zeros may disappear.

What is indexing?

Every character in a string has a position called an **index**.

String:	1	2	3	4	5
Index:	0	1	2	3	4

Python starts counting indexes from 0.

Example:

```
number = "12345"
print(number[0])
```

Output:

1

What is negative indexing?

Negative indexing counts characters from the end of the string.

String:		1	2	3	4	5
Negative index:	-5	-4	-3	-2	-1	

Example:

```
number = "12345"
print(number[-1])
```

Output:

5

-1 means the last character.

What is slicing?

Slicing allows you to take multiple characters from a string.

Basic syntax:

```
string[start:end]
```

The start position is included, but the end position is not included.

Example:

```
word = "Python"
print(word[0:3])
```

Output:

```
Pyt
```

Getting the last four characters

You can use negative slicing to get the last four characters:

```
number[-4:]
```

Here:

- -4 means start from the fourth character from the end.
- The empty space after : means continue until the end.

Finding string length

Use `len()` to count the number of characters.

```
number = "123456"  
  
print(len(number))
```

Output:

```
6
```

Repeating a character

Python can repeat a string using `*`.

```
print("X" * 4)
```

Output:

```
XXXX
```

This is useful for creating the masked part of the account number.

6. Easy Example

Consider masking a phone number by showing only its final two digits.

```
phone = "9876543210"  
  
last_two_digits = phone[-2:]  
hidden_part = "X" * 8
```

The two parts can then be joined:

```
XXXXXXXXX10
```

Important idea:

Number of X characters
=
Total string length - Number of visible characters

This example demonstrates slicing and masking, but it is not the complete solution to today's problem.

7. Problem Statement

Create a Python program that:

1. Asks the user to enter an account number.
2. Keeps the account number as a string.
3. Checks whether the account number is empty.
4. Checks whether it contains at least four digits.
5. Shows only the last four digits.
6. Replaces all earlier digits with X.

Example:

Enter account number: 1234567890
Masked account number: XXXXXX7890

8. Concepts Used

- Variable
 - `input()`
 - String
 - `len()`
 - if-else
 - Negative slicing
 - String repetition
 - String concatenation
 - Function
 - Parameter
 - Return value
-

9. Thought Process

Step 1: Read the account number

Ask the user to enter an account number.

Keep the input as a string because account numbers are identifiers, not values used for arithmetic.

Step 2: Remove accidental spaces

A user may enter spaces before or after the account number.

For example:

```
" 1234567890 "
```

Using `strip()` can remove those outside spaces.

Step 3: Check for empty input

After cleaning the input, check whether the account number is empty.

If it is empty, return or display an appropriate message.

Step 4: Check the length

Use `len()` to determine how many characters are present.

If the account number has fewer than four characters, there are not enough characters to safely apply the normal masking rule.

Step 5: Get the last four digits

Use negative slicing to select the final four characters.

Think about:

```
Start four positions from the end  
Continue until the end
```

Step 6: Calculate how many characters to hide

Subtract 4 from the total account-number length.

```
Hidden character count = total length - 4
```

For an account number with ten digits:

```
10 - 4 = 6
```

Therefore, six X characters are needed.

Step 7: Create the masked portion

Repeat "X" the required number of times.

Step 8: Join both portions

Combine:

```
masked portion + last four digits
```

Then return the final masked account number from the function.

10. Pseudocode

```
START

DEFINE a function that accepts account number

    REMOVE extra spaces from account number

    IF account number is empty
        RETURN an empty-input message

    IF account number length is less than 4
        RETURN a short-account-number message

    GET the last 4 characters using slicing

    CALCULATE hidden length:
        total length minus 4

    CREATE X characters based on hidden length

    COMBINE X characters with last 4 characters

    RETURN masked account number

ASK user to enter account number

CALL the masking function

PRINT the returned result

END
```

11. Suggested Solving Approach: Functional Approach

Create a function responsible only for masking the account number.

Possible function structure:

```
def mask_account_number(account_number):
    # Clean the input
    # Handle empty input
    # Handle account numbers shorter than 4 characters
    # Extract the last four characters
    # Create the masked portion
    # Return the masked account number
```

Outside the function:

```
# Ask for the account number
# Call the function
# Print the returned result
```

This approach separates the program into two responsibilities:

```
Input and output
    ↓
Masking function
```

Benefits:

- Easier to understand
- Easier to test
- Easier to reuse
- Keeps the masking logic organized

12. Easy Edge Cases

Edge Case 1: Account number shorter than four digits

Input:

```
123
```

Possible output:

```
Account number must contain at least 4 digits.
```

Do not try to create a negative number of X characters.

Edge Case 2: Empty account number

Input:

Possible output:

```
Account number cannot be empty.
```

Check the input after applying `strip()`.

Edge Case 3: Exactly four digits

Input:

```
1234
```

Expected behaviour:

```
1234
```

There are no earlier digits to replace because:

```
4 - 4 = 0
```

Edge Case 4: Leading zeros

Input:

0012345678

Expected masked output:

XXXXXX5678

This is why the account number should remain a string.

13. Expected Input

Example 1:

Enter account number: 1234567890

Example 2:

Enter account number: 987654321234

Example 3:

Enter account number: 123

14. Expected Output

For input:

1234567890

Output:

Masked account number: XXXXXX7890

For input:

987654321234

Output:

Masked account number: XXXXXXXX1234

For input:

123

Output:

Account number must contain at least 4 digits.

15. Hint Only

Use these three ideas:

```
len(account_number)
```

```
account_number[-4:]
```

```
"X" * hidden_length
```

Calculate the hidden length using:

```
total account-number length - 4
```

Then join the hidden portion with the last four characters.